

Rainfall Patterns and 30-Minute Rainfall Prediction with Apache Spark

DSA5208 | Scalable Distributed Computing, Project 2

ADITYA VIJINDRA PANDHARI | A0328872R
APURWA KUMARI | A0333260R
VANSHIKA DHAMIJA | A0333989H

Date: 17 October 2026

Abstract: This report analyses eight years (2017–2024) of five-minute rainfall readings from Singapore's gauge network (48.5 million records from 91 stations) using Apache Spark. Task 1 explores how rainfall varies within a year and from year to year. Task 2 builds a model that estimates the probability that a station records more than a chosen amount of rain in the next 30 minutes, taking the station, the time and the threshold as inputs. After a data-quality stage that caught two kinds of faulty gauge that a range check cannot see, we built an interpretable logistic-regression baseline and then a cross-validated **random forest** on richer weather features. On the untouched 2024 test year the final model reaches a PR-AUC of **0.465** at the 5 mm threshold (55× the event rate) with ROC-AUC **0.969**, beats the logistic baseline at every threshold on identical rows, and gives calibrated probabilities. An earlier version of the complete notebook was also executed on a Google Cloud Dataproc cluster with identical conclusions, and doubling the cluster's workers halved the run time of the core Spark job.

1. Executive Summary

This project processes the data.gov.sg **Historical Rainfall across Singapore** collection (2017–2024, 7.5 GB of CSV) with Apache Spark, from raw files to a trained model, in one notebook. Spark handles every step that touches the 48.5 million readings: parsing, cleaning, the window-function features, the training sets and the MLlib models.

Task 1 shows that Singapore's rainfall has two wet seasons, two daily rhythms, a strong west–east gradient and large year-to-year swings driven by how often it rains rather than how hard. **Task 2** delivers a single Spark MLlib model that answers P(rain in the next 30 minutes > h) for any station, time and threshold h. The final model is a **random forest** chosen by year-blocked cross-validation. On identical 2024 rows it beats the Stage 1 logistic model on PR-AUC, ROC-AUC and log loss at every threshold.

Model	Mean PR-AUC	Mean ROC-AUC	Mean log loss	PR-AUC at 5 mm	ROC-AUC at 5 mm
Final model: random forest (Stage 2)	0.450	0.973	0.0266	0.459	0.970
Stage 1: logistic regression	0.408	0.921	0.0328	0.418	0.912
Climatology baseline	0.023	0.695	0.0553	0.018	0.684

Table 1. Final model against the Stage 1 model and the climatology baseline on identical 2024 rows (1,255,003 rows per threshold; means over the 1, 2, 5, 10 and 20 mm thresholds).

Key Insight

The most useful information for the next 30 minutes is **what is raining right now, nearby**: the station's own rain in the last 5–15 minutes, how long it has been dry, and the rain at gauges within 10 km. Time of day and season help, but on their own they are barely better than climatology (Model A, Section 8). A model that cannot see any rain observation has almost no skill for this task. A model that can see the neighbourhood is 55 times better than chance at 5 mm.

2. Problem Statement & Success Metrics

Task 1. Explore the weather patterns in the rainfall record and how they change within a year and across 2017–2024, with analysis and visualisations.

Task 2. Build a model that estimates the probability that a monitoring station will accumulate more than a given amount of rain in the next 30 minutes. Inputs: station, prediction time and threshold h. The model must be built and evaluated with Spark MLlib.

2.1 Why accuracy is the wrong metric

Rain above 5 mm in a half-hour happens in only **0.83 %** of station-times. A model that always answers "no" is 99.16 % accurate and completely useless. We therefore judge the probabilities themselves:

Metric	Question it answers	Good value
PR-AUC (average precision)	When the model flags rain, how often is it right, across all cut-offs? Compare with the event rate.	high
ROC-AUC	How well does it rank wet half-hours above dry ones?	high
Log loss	How much confident wrong probabilities are penalised (a proper scoring rule; the cross-validation metric)	low
Brier skill score	Share of the climatology baseline's squared error that the model removes	high
Reliability	Do half-hours forecast at 20 % actually turn wet 20 % of the time?	on the diagonal
Monotonicity	P(> 10 mm) must never exceed P(> 5 mm) for the same station and time	0 violations

Table 2. Evaluation metrics used throughout. All are computed on the 2023 validation year for model choice and once on the 2024 test year.

2.2 One model for every threshold

> **Design choice:** the threshold h is an **input feature** of a single binary classifier. Each training example (station, time) is paired with several thresholds, each with its own 0/1 label, so the model learns how the probability falls as h rises and can answer for any h in its trained range (0.2–60 mm). This replaces one model per threshold with one model for all.

3. Dataset & Computing Environment

The collection has one CSV per year with one row per station per five-minute reading. The eight files total **7.5 GB and 48.5 million rows** from 91 distinct stations. Between 54–80 stations report in any one year because the network grew in 2020 and gauges come and go.

Column group	Columns
Time	timestamp (ISO 8601, +08:00), update_timestamp, reading_update_timestamp, date
Station	station_id, station_name, station_device_id, location_longitude, location_latitude
Measurement	reading_value (mm per 5 minutes), reading_type, reading_unit

Table 3. Dataset schema. 12 columns, one row per station per 5-minute interval.

3.1 Computing environment

The submitted run uses Apache Spark in local mode on one laptop. On **Google Cloud Dataproc**, the core Spark job was timed on 2 and 4 workers to measure horizontal scaling (Section 8.5), and the complete notebook was executed on a 2-worker cluster with Spark on YARN and the data on HDFS (Section 8.6).

Item	Value
Machine	Apple MacBook Pro, Apple M4 Max, 16 cores (12 performance + 4 efficiency), 128 GB RAM, macOS
Spark	Apache Spark 3.5.5 (PySpark), master local[14], driver memory 16g, 96 shuffle partitions
Software	Python 3.11.17, OpenJDK 17, pandas / NumPy / Matplotlib 3.0.6 / 1.26.4 / 3.11.2
Storage layers	CSV 7.5 GB → bronze Parquet 49 MB (typed raw copy) → silver 413 MB (clean readings) → gold 5.9 GB (features, training sets, predictions); Snappy-compressed, partitioned by year
Saved models	final random forest 11.5 MB; Stage 1 logistic baseline 13 KB
Cloud: scaling runs	Google Cloud Dataproc 2.2 (Debian 12, Spark 3.5), region asia-southeast1 (Singapore); master and workers n2-standard-2 (2 vCPU, 8 GB), 50 GB standard disks; 2 and then 4 workers; data in a Cloud Storage bucket; gcloud dataproc jobs submit pyspark (cloud/dataproc_benchmark.py)
Cloud: complete run	Dataproc 2.2, master + 2 workers n2-highmem-4 (4 vCPU, 32 GB, Intel Xeon 2.8 GHz), 100/75/75 GB balanced persistent disks (the education project's quotas: 12 vCPUs, 250 GB SSD); Spark 3.5.3 on YARN, data layers on HDFS, driver 16 GB; Cloud NAT for package installation

Table 4. Computing environment: the laptop as recorded by the notebook (t00_environment.csv), the cluster as configured for the Dataproc jobs.

Stage	Minutes
Ingestion and data-quality audit	0.4
Cleaning and pre-processing	0.6
Task 1 analyses	0.5
Stage 1 features, sets, baselines and models	7.9
Stage 2 features and feature screening	1.2
Stage 2 cross-validation (25 settings × 4 folds)	37.4
Stage 2 ablation, calibration, refit and 2024 evaluation	24.5
Total	**72.6**

Table 5. Wall-clock time of the notebook's stages on this machine in the submitted run (t40_stage_timings.csv).

4. Data Cleaning

4.1 What the audit found

- **Values are clean:** no missing timestamps, station IDs or readings; no negative values; one reading type in one unit (mm). A single reading of 37.2 mm in five minutes, far above every other value, is an instrument fault.
- **Timestamps are not:** 1.78 million readings, all in 2017, are stamped at hh:m4:59 instead of hh:m5:00. Rounding to the 5-minute grid fixes this and reveals 86 duplicate station-slots.
- **Faulty gauges hide in plausible values:** a broken gauge reports 0.0 mm through a wet month, and one gauge reported a 749 mm "day" while its five neighbours saw at most 32 mm. A range check catches

neither. Both are caught by comparing a gauge with its neighbours (67 dead station-months and 4 spike days).

- **What a timestamp means is undocumented.** Readings are published 2–10 minutes after their stamp, so we take a stamp as the start of its 5-minute interval. This is the conservative reading for Task 2: no reading *measured* after the prediction time can reach the features. It is not a real-time guarantee. At clock time t the reading stamped t − 5 min may not have been published yet, and the gauge-fault rules below judge a whole month or day, which is only possible in hindsight (Section 6).

4.2 Cleaning steps

Cleaning step	Records remaining	Change
Raw records (bronze)	48,538,713	—
After the range check (one 37.2 mm reading removed)	48,538,712	-1
After de-duplication of station-slots	48,538,626	-86
After removing dead-gauge station-months	48,340,082	-198,544
After removing spike station-days (final, silver)	48,338,938	-1,144

Table 6. Records remaining after each cleaning step. 99.59 % of the raw records are kept.

Note on the gauge-fault rules. They remove only 0.4 % of the readings but change the conclusions: left in, the S107 fault would have shown up in Task 1 as a record 147 mm hour in 2022, and the dead months would pull island-wide means down. The rules are deliberately simple and documented: a station-month is dead if its rain rate is below 10 % of the network median when that median is at least 50 mm. A station-day is a spike if it exceeds 100 mm and 4× the wettest gauge within 5 km.

Missing readings are never filled with zeros: a missing reading is unknown, not dry. Later steps use completeness rules instead (a station-day needs 80 % of its slots, an island-day at least 20 usable gauges, and a fixed 21-gauge network checks that the changing network does not distort the results).

De-duplication keeps the most recently updated reading of each station-slot with a single max over an (update time, value) struct. This needs one shuffle instead of a sorted window over 48 million rows. The physical plan printed by the notebook confirms the single exchange. Because a max over a struct cannot be computed by Spark's hash aggregate, Spark sorts each partition locally and uses SortAggregate. The sort happens inside each partition, with no global sort:

```
+- SortAggregate(key=[station_id, ts], functions=[max(struct(upd, upd, rain_mm, rain_mm)),
count(1), …
   +- Sort [station_id ASC NULLS FIRST, ts ASC NULLS FIRST], false, 0
      +- Exchange hashpartitioning(station_id, ts, 96)
         +- SortAggregate(key=[station_id, ts], functions=[partial_max(struct(upd, upd, rain_mm,
rain_mm)), …
            +- Sort [station_id ASC NULLS FIRST, ts ASC NULLS FIRST], false, 0
```

5. Weather Patterns (Task 1)

All Task 1 quantities are Spark aggregations of the cleaned readings, built from station-days with completeness rules so that the growing gauge network cannot masquerade as a change in climate.

5.1 Year to year: dry years have fewer rain events, not lighter rain

Year	Rainfall (mm)	Wet days (≥ 1 mm)	mm per wet day	Events per gauge	Gauges
2017	2,454	239	10.3	403	58
2018	2,656	245	10.9	374	56
2019	1,787	172	10.4	266	54
2020	2,631	230	11.4	365	80
2021	2,900	214	13.6	381	74
2022	2,893	241	12.0	437	72
2023	2,879	228	12.6	412	73

Year	Rainfall (mm)	Wet days (≥ 1 mm)	mm per wet day	Events per gauge	Gauges
2024	2,708	233	11.6	397	70

Table 7. Annual island-wide rainfall (gap-filled), wet days scaled to a full year, and rain events per gauge.

Annual rainfall ranges from **1,787 mm (2019) to 2,900 mm (2021)** around a 2,613 mm mean. 2019 was dry because it rained less often: 172 wet days against 233 in the other years. Its rain per wet day (10.4 mm) was close to normal (11.8 mm). Dry and wet spells last whole seasons: the 2019 dry spell was 12 consecutive below-average months.

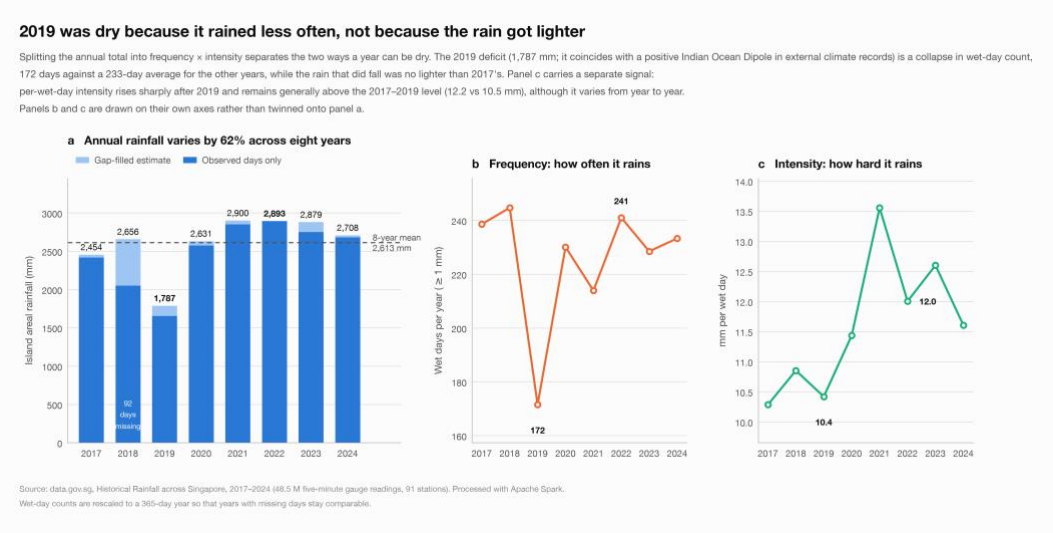


Fig 1 – Annual rainfall (a) split into frequency (b, wet days) and intensity (c, mm per wet day).

5.2 Within a year: two wet seasons and two daily rhythms

The monthly climatology has two wet seasons, **November–January** (north-east monsoon) and **April–May** (inter-monsoon): the wettest months are Nov (315 mm), Dec (261 mm), Apr (248 mm) and the driest are Jul (147 mm), Feb (145 mm). The same calendar month can differ tenfold between years, so the calendar alone predicts little. Within the day, Singapore has two rainfall clocks (Fig 2): from June to September the mean rain rate peaks at about **10:00**, consistent with squall lines arriving in the morning, while in every other season it peaks sharply at **15:00–16:00** with afternoon convection. Rain is also extremely concentrated: the wettest 10 % of wet five-minute slots deliver **49 %** of a year's rain and the wettest 1 % deliver 10 %. The typical rain event is short and light (median 20 minutes, 1.0 mm).

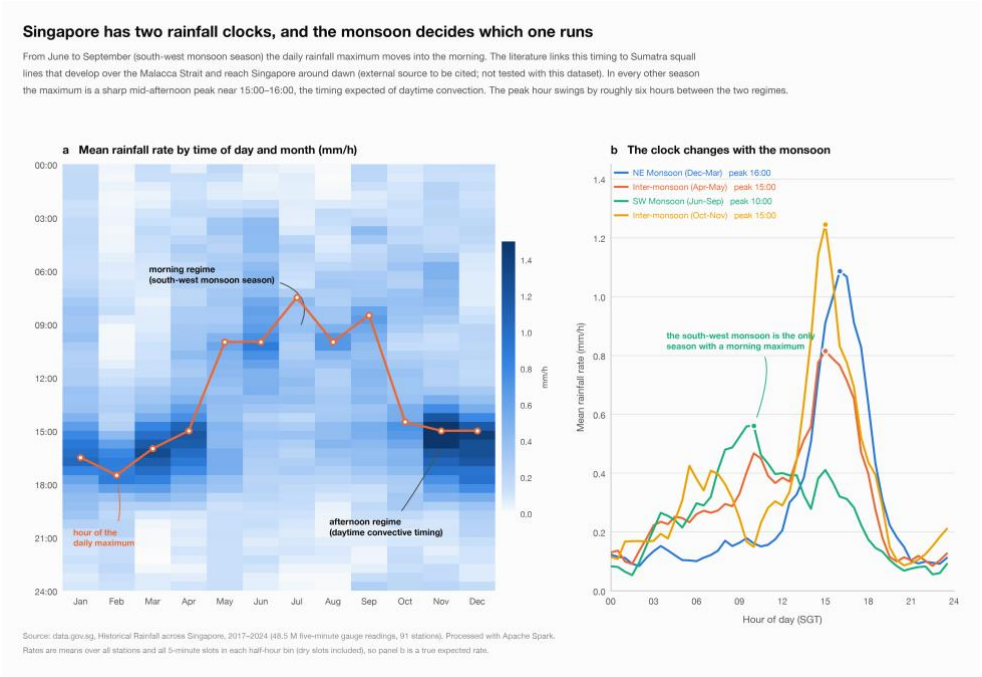


Fig 2 – Mean rainfall rate by half-hour and month (a) and by season (b): a 10:00 peak in the south-west monsoon, a 15:00–16:00 peak otherwise.

5.3 Across the island: 2× wetter in the west

Over 2021–2024, the 53 gauges with complete records average from **3,548 mm a year (S228) down to 1,700 mm (S208)**, a factor of 2.1 across an island 50 km wide. Rainfall falls by about 266 mm per 10 km travelled east (r = -0.68). Rain is also local: gauges less than 2 km apart agree closely on their 30-minute totals (correlation 0.84) but beyond 20 km they are nearly independent (0.15), and the correlation drops to 1/e at about **8.5 km**. A single island-wide forecast cannot be right everywhere, so Task 2 predicts per station and uses neighbouring gauges within 10 km.

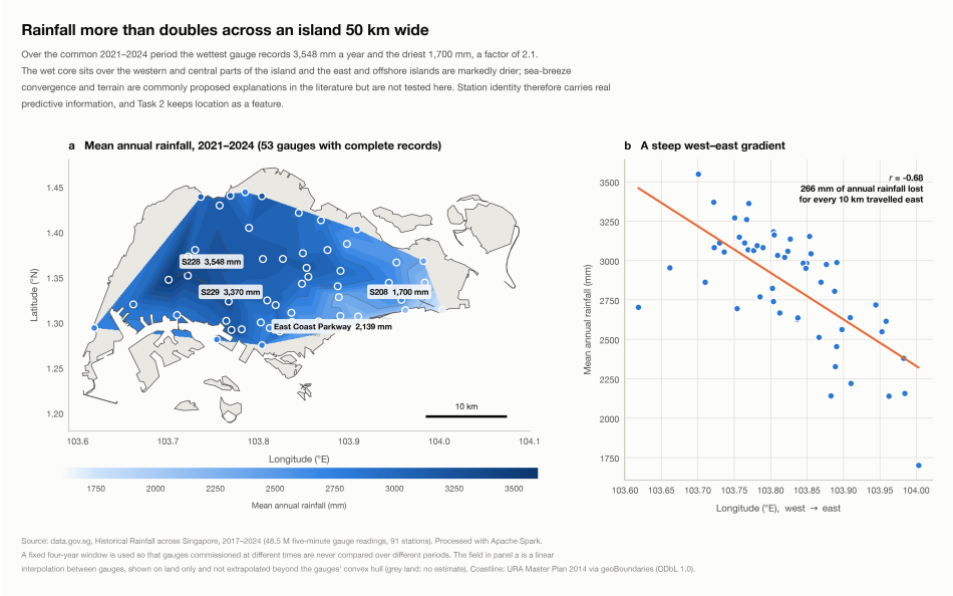


Fig 3 – Mean annual rainfall 2021–2024 interpolated between gauges on Singapore's land area (a), and its west–east gradient (b).

5.4 Heavy rain and trends

Heavy half-hours (≥ 5 mm in 30 minutes) were **about 23 % more frequent in 2020–2024 than in 2017–2019** (20 % on the fixed 21-gauge network, so the 2020 network expansion explains little of it). With only eight annual values, however, an exact Mann–Kendall test finds **no statistically significant trend in any series** (Table 8): the period is too short to separate a trend from year-to-year variability, and "not significant" means "not shown", not "no change".

Annual series	Kendall τ	p-value	Sen slope (% of mean per year)
annual rainfall, gap-filled (mm)	+0.36	0.28	+2.3 %
wet days per year (≥ 1 mm, scaled)	-0.07	0.90	-0.3 %
intensity (mm per wet day)	+0.57	0.06	+2.7 %
rain events per gauge	+0.29	0.40	+1.6 %
30-min totals ≥ 5 mm, rate (all gauges)	+0.43	0.18	+2.4 %
30-min totals ≥ 5 mm, rate (fixed network)	+0.36	0.28	+1.4 %
annual max 30-min depth (mm, any gauge)	-0.36	0.28	-1.2 %
annual max 60-min depth (mm, any gauge)	+0.29	0.40	+1.5 %

Table 8. Mann–Kendall trend tests on the eight annual values (exact two-sided p-value) with Sen's slope.

Check on a fixed network. Re-deriving the annual and monthly results on the fixed network of 21 gauges that report in all eight years gives annual totals that correlate at r = 0.986 with the full network, the same driest year and wettest month, and monthly anomalies that agree in sign 97.9 % of the time.

What Task 1 told Task 2: use the recent rainfall state, not only the calendar (spells persist); include hour of day *and* day of year (two daily regimes switched by the monsoon); predict an exceedance probability, not an amount (heavy-tailed intensity); use station location and neighbours within 10 km (gradient and 8.5 km correlation length); and evaluate with imbalance-aware metrics (base rate 0.83 % at 5 mm, varying from 0.13 % to 3.07 % with hour and season).

6. Feature Engineering

Every feature is computed with Spark window functions over each station's time series, using **time-based windows** (rangeBetween on epoch seconds) with an explicit count of the readings present, so a gap in the data never silently pulls in older readings. Neighbour features come from a small broadcast table of station pairs built from coordinates only, joined to the readings and grouped by (target station, slot), so the 48 million-row table is never joined station-by-station.

> **Key design principle:** the model uses only information available at the prediction time t: the station's coordinates, the clock, and rain stamped at or before t − 5 min at the station and its neighbours. An independent recomputation in plain pandas of 300 random rows, from readings truncated at the anchor, matched every Spark feature and target exactly, for both stages. Neighbour denominators count only gauges that have reported *by* the prediction time (an earlier version counted gauges that reported later that day; the check exposed this look-ahead). **Two limits of this guarantee.** The checks run on the cleaned data, so they verify the time boundary of the window computations. They do not show that every input was *published* by t (readings arrive 2–10 minutes after their stamp), nor that the cleaning decisions could have been made live (the dead-gauge and spike rules use a whole month or day). An operational version would use a larger lag (anchor t − 10 or t − 15 min, with the same features and a retrained model) and causal quality rules.

Group	Features	Why (Task 1 evidence)
Inputs	threshold h as log(1 + h) (Stage 1) or h and log h (Stage 2); station	the task's own inputs
Time	sine/cosine of time of day and day of year; season	two daily regimes, monsoon seasons
Location	latitude, longitude	2× west–east gradient
Recent rain at the station	last 5 min; last 15/30/60/180-min totals; wet slots and peak in the last hour; **storm trend** (last 15 min minus the 15 before); **minutes since the last rain**; **readings present** in the last 1 h / 3 h; **rain so far today**	events last ~20 min; spells persist; missing data is a feature, not zero rain
Neighbouring gauges	Stage 1: up to 4 within 10 km (mean and max of last 30 min, mean of last 60 min, latest 5-min max). Stage 2: **three rings** (≤ 5 km, ≤ 10 km and 5–20 km to the **west**), each with mean, max and share wet in the last 30 min, plus availability flags	correlation length ~8.5 km; squall lines arrive from the west
Island-wide	**share of all gauges wet** in their last 30 min and their mean rain	wet days vs dry days

Table 9. Feature groups. Stage 1 uses the compact set; Stage 2 adds the richer items in bold and lets cross-validation choose.

Stage 1 keeps rows with a complete 3-hour history and all six future slots (32,962,257 training prediction times). Stage 2 keeps a row even when its history has gaps, because the number of readings present is itself a feature. It takes prediction times every 15 minutes in 2017–2023 (neighbouring 5-minute rows are near-duplicates) and every 5 minutes in the 2024 test year.

7. Modelling Approach

7.1 Chronological split and year-blocked cross-validation

> **No random splits.** Consecutive five-minute examples share five of their six future slots, so a random split would leak the answer. Stage 1 trains on 2017–2022, selects on 2023 and is tested once on 2024. Stage 2 uses Spark's CrossValidator with a fold column of **whole years** (2017–18, 2019–20, 2021–22, 2023), refits on 2017–2023, and is tested once on 2024. The test year is never used for any choice. The final 2024 boundary is purged (Section 8.1). The three boundaries between folds are not: about 369 of the 13.7 M prediction rows, all at 23:30 and 23:45 on a 31 December, have target windows that reach up to 20 minutes into the next fold. This weakens the isolation between folds slightly, but it is real.

7.2 Training sets

The two stages sample differently, and the difference matters for calibration. **Stage 1** keeps a row for threshold h if a hash of (station, time, h) falls below a rate that depends only on h (about 8,000 positives per threshold). Because the rule never looks at the label, the kept rows are a plain random sample within each threshold, the positive rate equals the population rate, and no weighting is needed. **Stage 2** keeps every row with rain in the last 3 hours *or in

the next 30 minutes* and only 3 % of the remaining quiet rows. That rule looks at the outcome, so the sample over-represents rain. Inverse inclusion weights make it unbiased again (a Horvitz–Thompson estimator). The weight is 1 for the always-kept rows and 1 / 0.03 ≈ 33.3 for the kept quiet rows, and MLlib's learners and evaluators apply it through weightCol. The notebook verifies that the weighted positive rate equals the population rate. Each Stage 2 row is then paired with three thresholds drawn log-uniformly from 0.2–60 mm, independently of the outcome.

7.3 Models evaluated

Candidate	PR-AUC	ROC-AUC	Log-loss skill	Threshold violations
Climatology (season × hour × threshold)	0.043	0.720	0.000	—
Base rate per threshold	0.018	0.500	-0.059	—
Logistic: own gauge only	0.403	0.899	0.325	0.00 %
Logistic: + 4 neighbouring gauges	0.445	0.929	0.373	0.00 %
Logistic: + threshold interactions	0.458	0.929	0.393	0.00 %
Logistic: + interactions, stronger L2	0.463	0.930	0.383	0.00 %
GBT, depth 5 (80 trees)	0.452	0.925	0.029	0.03 %
GBT, depth 7 (80 trees)	0.449	0.933	0.031	0.22 %

Table 10. Stage 1 candidates: Spark ML pipelines (StringIndexer → OneHotEncoder → VectorAssembler → classifier) fitted on 2017–2022 and scored on 2023, macro-averaged over six thresholds. Selection rule, fixed in advance: highest log-loss skill.

Each step of the logistic model adds skill: neighbouring gauges lift PR-AUC from 0.40 to 0.44 and the threshold interactions lift log-loss skill to the best of all candidates. The Stage 1 GBTs rank about as well but their probabilities are poor (log-loss skill 0.03) because Spark's GBT starts every prediction at 0.5 and moves by about the step size per tree, so 80 trees cannot reach the 0.1 % probabilities the 20 mm threshold needs. This is why Stage 2 adds a calibrator.

Model family	Settings tried	Best setting	CV log loss (mean ± sd over folds)
Logistic regression	6	regParam=0.001, elasticNetParam=0	0.0391 ± 0.0022
Random forest	4	numTrees=100, maxDepth=12	0.0319 ± 0.0018
Gradient-boosted trees	12	maxDepth=4, maxIter=100, stepSize=0.1	0.0403 ± 0.0018

Table 11. Stage 2: best cross-validated setting per model family (4 year-blocked folds, weighted log loss, 22 settings in total). The selector size k was tuned first with logistic regression (k=33: 0.0425, k=16: 0.0454, k=8: 0.0585); all 33 candidates were kept.

7.4 Calibration and consistency

Tree probabilities are not automatically calibrated. A Platt-style calibrator (an MLlib logistic regression on logit p, log h and their product) is fitted on the first half of 2023 and judged on the second half. It is kept only if it lowers the held-out log loss. For the random forest it was kept (held-out log loss 0.0294 → 0.0288); for the GBT it reduced the held-out log loss from 0.0381 to 0.0311. A tree model also does not guarantee that P(> 10 mm) ≤ P(> 5 mm). So, for each station and time, the probabilities at the six evaluated thresholds are made non-increasing with a running minimum, which changed at most 0.055 % of the 2024 predictions. The delivered predict() function applies the same correction: it scores the same six thresholds and applies the same running minimum, so it returns exactly the evaluated probability at any of them. For any other threshold it interpolates in log h between those six points. The curve is extended to 0.2 mm and to 30 and 60 mm from the model's values there and clipped so that it never increases. The reported metrics therefore describe the same probability function that the predictor delivers. The Stage 1 logistic model needs no correction: it showed no violation in the 25,771,260 adjacent threshold pairs checked, and monotonicity is guaranteed whenever its threshold coefficient (−1.23 on log(1 + h)) outweighs the two positive threshold interactions, which holds for every physically valid input because a 30-minute total cannot exceed the 180-minute total and a neighbour maximum cannot exceed four times the neighbour mean.

8. Results

8.1 The final random forest on every 2024 row

The final model was refitted on 2017–2023 (with the end-of-2023 rows whose target window reaches into 2024 dropped) and scored on **every station and five-minute time of 2024** (6,371,188 prediction times × six thresholds),

with no sampling and no weights. Precision and recall use the F1-optimal cut-off chosen on 2023 and applied unchanged.

h (mm)	Event rate	PR-AUC	ROC-AUC	Log loss	Brier skill	Cut-off	Precision	Recall	F1
0.5	3.63 %	0.721	0.965	0.0645	0.492	0.40	0.71	0.59	0.65
1	2.57 %	0.667	0.966	0.0516	0.448	0.35	0.65	0.58	0.61
2	1.69 %	0.587	0.966	0.0395	0.386	0.25	0.57	0.54	0.55
5	0.84 %	0.465	0.969	0.0240	0.293	0.20	0.48	0.46	0.47
10	0.38 %	0.351	0.973	0.0130	0.211	0.15	0.42	0.38	0.40
20	0.10 %	0.185	0.982	0.0045	0.101	0.10	0.26	0.28	0.27

Table 12. Final model on the 2024 test year by threshold. Brier skill = share of the climatology error removed.

At the 5 mm threshold (event rate 0.84 %) the model reaches **PR-AUC 0.465**, 55 times the event rate, with ROC-AUC 0.969, and removes 29 % of the climatology error. At its 2023 cut-off it catches 46 % of the events with 48 % precision. Skill falls as events get rarer (PR-AUC 0.721 at 0.5 mm, 0.185 at 20 mm) while ROC-AUC stays above 0.96 throughout. Calibration at 5 mm is good on average but not uniform. Over the bins below 10 %, which hold 98.6 % of all forecasts, the observed frequency is only 0.12 points above the forecast, and the expected calibration error is 0.0016. That average is dominated by the dry rows. Bin by bin, the model under-forecasts between 2 % and 35 % (worst bin: 14.0 % forecast, 17.6 % observed) and over-forecasts between 45 % and 75 % (65.0 % forecast, 60.8 % observed). Fig. 20 of the notebook shows the full curve.

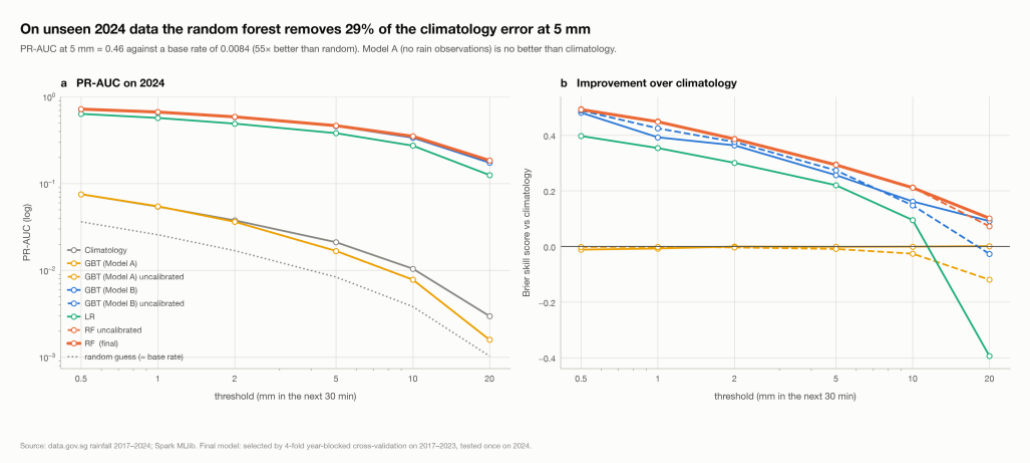


Fig 4 – PR-AUC (a) and Brier skill against climatology (b) on all of 2024 for every Stage 2 model. Model A, which sees no rain observation, is no better than climatology.

8.2 Model families at 5 mm

Model	PR-AUC	ROC-AUC	Log loss	Brier skill
Climatology (station × month × hour)	0.021	0.679	0.0483	0.000
GBT, Model A: threshold + time + station only	0.017	0.684	0.0473	-0.002
Logistic regression (tuned)	0.381	0.951	0.0289	0.220
GBT, Model B (tuned, calibrated)	0.460	0.966	0.0263	0.256
Random forest, uncalibrated	0.465	0.969	0.0238	0.295
Random forest, calibrated	0.465	0.969	0.0240	0.293

Table 13. Stage 2 models on all 2024 rows at the 5 mm threshold. Model A has only the threshold, time and station.

8.3 Head-to-head on identical rows

The two stages evaluate different row sets, so for a like-for-like comparison both models were scored on the 1,255,003 Stage 1 evaluation rows per threshold, matched on station, time and threshold. The matched labels agree exactly. The final model wins on PR-AUC at 5 of 5 thresholds, on ROC-AUC at 5 of 5 and on log loss at 5 of 5. The largest gain is in ranking: knowing how long it has been dry, whether it is raining elsewhere on the island and whether a storm is building tells the model in advance which dry half-hours will turn wet.

h (mm)	Events	PR-AUC clim.	PR-AUC Stage 1	PR-AUC final	ROC-AUC Stage 1	ROC-AUC final	Log loss Stage 1	Log loss final
1	32,509	0.049	0.585	0.663	0.918	0.967	0.0661	0.0521
2	21,290	0.033	0.524	0.587	0.915	0.968	0.0489	0.0395
5	10,445	0.018	0.418	0.459	0.912	0.970	0.0287	0.0238
10	4,790	0.010	0.327	0.352	0.917	0.975	0.0150	0.0129
20	1,323	0.004	0.186	0.191	0.941	0.982	0.0050	0.0046

Table 14. Stage 1 logistic model vs the final model on identical 2024 rows.

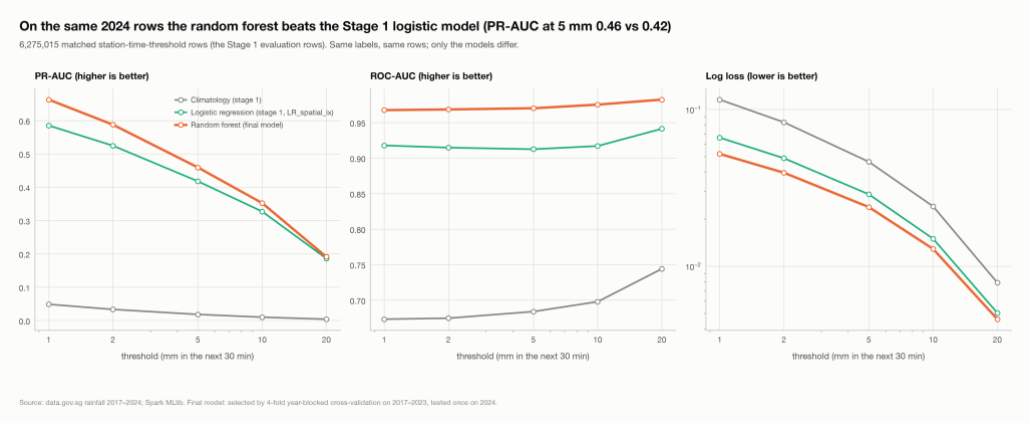


Fig 5 – PR-AUC, ROC-AUC and log loss by threshold for the climatology, the Stage 1 logistic model and the final model on identical 2024 rows.

8.4 Strong and weak scaling on the laptop

We timed the main part of the cleaning stage: read the bronze rows, snap the timestamps, de-duplicate (one shuffle) and aggregate to station-days (a second shuffle). Each configuration ran in a fresh Spark session, and each time is the median of three runs after a warm-up. **Strong scaling** keeps the problem fixed (all 8 years) and adds cores: 16 cores run the job **4.9× faster than 2** (28 s → 6 s), a parallel efficiency of 61 %. **Amdahl's law**, Speedup = 1 / (S + P/n), explains why the gain flattens: fitting T(n) = a + b/n to the timings gives T(n) = 1.79 + 51.1/n seconds, so the serial part S is **3.4 %** of the one-core time. Serial work is work that does not divide across cores, such as task scheduling, the shuffle exchange and the final single-row result. Under that fitted model the ceiling is 30× over one core (15.3× over two). This is an extrapolation of a two-parameter fit to four points (RMSE 0.65 s), not a measurement.

Cores	Wall time (s)	Speed-up vs 2 cores	Amdahl fit	Ideal	Parallel efficiency
2	27.8	1.00×	1.00×	1×	100 %
4	13.6	2.04×	1.88×	2×	102 %
8	8.0	3.48×	3.34×	4×	87 %
16	5.7	4.88×	5.49×	8×	61 %

Table 15. Strong scaling: all 8 years on 2–16 cores (median of three runs; t42_scaling.csv, t47_amdahl_fit.csv). An efficiency slightly above 100 % is run-to-run noise in the medians.

Weak scaling grows the problem with the cores (1 year on 2 cores, 2 on 4, 4 on 8, 8 on 16), so ideally the time stays constant and the throughput grows linearly with the cores (**Gustafson's law**). The time rises only from 3.3 s to 5.8 s while the data grows 9.2-fold, a scaled speed-up of 5.3× on 16 cores (ideal 8×, efficiency 66 %). Both experiments point to the same limit: the cores of one machine share one memory bus and one SSD, and 4 of the 16 are slower efficiency cores. A cluster avoids this because each node has its own CPU, memory and disk (Section 8.5).

Cores	Years	Rows (M)	Wall time (s)	Scaled speed-up	Ideal	Efficiency
2	2017	5.3	3.3	1.00×	1×	100 %
4	2017-2018	9.9	3.5	1.77×	2×	88 %
8	2017-2020	21.2	3.9	3.41×	4×	85 %
16	2017-2024	48.5	5.8	5.25×	8×	66 %

Table 16. Weak scaling: the data grows with the cores (median of three runs; t45_weak_scaling.csv). Efficiency = throughput per core relative to 2 cores.

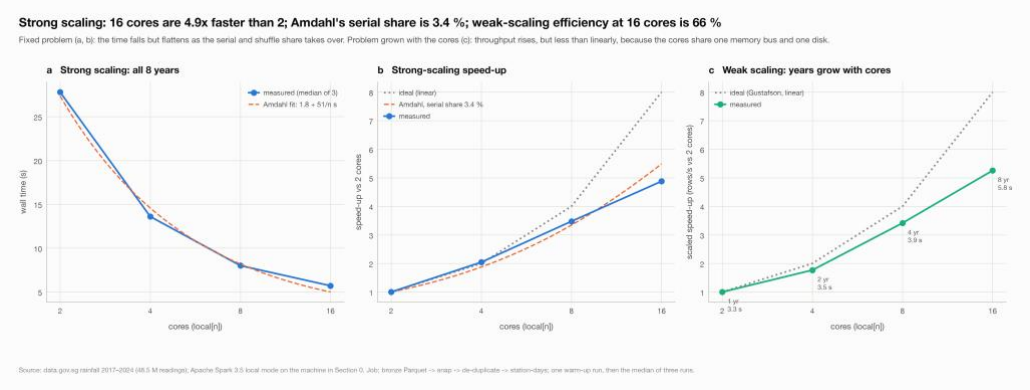


Fig 6 – Strong scaling: wall time with the Amdahl fit (a) and speed-up (b); weak scaling: scaled speed-up when the years grow with the cores (c).

8.5 The same job on a Google Cloud Dataproc cluster

The identical job was submitted to a Dataproc cluster with 2 and then 4 workers (n2-standard-2), reading the bronze layer from Cloud Storage. It returned exactly the same answer as the laptop. Ingesting the 7.5 GB of CSV from the bucket into bronze Parquet took 328 s on 2 workers. For the cleaning job, **doubling the workers cut the time from 74.8 s to 37.3 s, a 2.0× speed-up**, whereas on the laptop going from 4 to 8 cores gave 1.7× (13.6 s → 8.0 s).

Platform	Configuration	Cores	Wall time (s)	Speed-up	Efficiency
Laptop	local[2]	2	27.8	1.00×	100 %
Laptop	local[4]	4	13.6	2.04×	102 %
Laptop	local[8]	8	8.0	3.48×	87 %
Laptop	local[16]	16	5.7	4.88×	61 %
Dataproc cluster	2 workers x n2-standard-2	4	74.8 (73.4-76.1)	1.00×	100 %
Dataproc cluster	4 workers x n2-standard-2	8	37.3 (33.9-43.7)	2.01×	100 %

Table 17. The cleaning job on the laptop and on the Dataproc cluster (cluster: mean of three timed runs, range in brackets; t44_local_vs_cloud.csv).

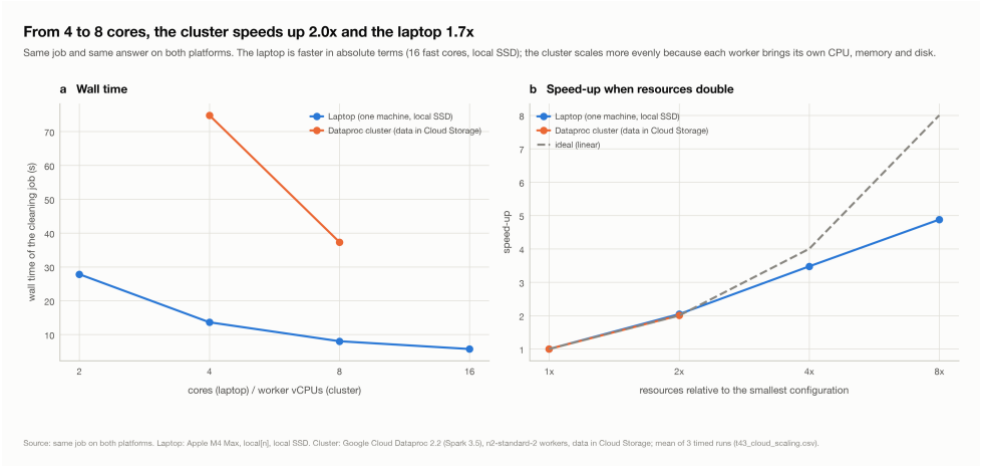


Fig 7 – Wall time (a) and speed-up when resources double (b): laptop cores against Dataproc workers for the same job.

What this shows. The laptop is about 5× faster than the cluster at the same core count: it has fast cores, 128 GB of memory and a local SSD, while each n2-standard-2 worker has 2 vCPUs, reads its input from Cloud Storage over the network (each request took 0.3–0.5 s in the job logs) and spills shuffles to standard disks. But adding cores to one machine shares one memory bus and one disk, so its efficiency falls (87 % at 8 cores, 61 % at 16), while adding workers adds CPU, memory and disk together, so the cluster scaled almost perfectly from 2 to 4 workers.

Conclusion: at 7.5 GB a single strong machine is the fastest and cheapest option. A cluster pays off once the data or the model no longer fits on one machine, and its near-linear scaling is what lets it grow. The education project's 12-vCPU quota limited the experiment to 4 workers.

8.6 The complete pipeline on the Dataproc cluster

Finally we executed the **whole notebook** on a Dataproc cluster: cleaning, Task 1, both stages of Task 2 with the full cross-validation, the 2024 evaluation and predict(). The cluster had a master and 2 workers n2-highmem-4 (4 vCPU, 32 GB; the 12-vCPU quota) and ran Spark 3.5.3 on YARN with the data layers on HDFS. The code is the notebook **as it stood on 14 October 2026**, converted by cloud/make_dataproc_notebook.py (only the Spark master, the storage and the laptop core benchmark change). That version predates four later revisions: the Spark-plan cell, the strong/weak-scaling rewrite, the Stage 2 train/test boundary purge and the fixed-grid predict(). The executed copy and a note on these differences are in cloud/full_run/. It ran without errors in 290 minutes of timed work against 73 on the laptop, 4.0× slower, for the same reasons as in Section 8.5: 8 executor cores on 2.8 GHz Xeons whose shuffles cross the network, against 14 fast cores that share one memory.

Stage	Laptop (min)	Cluster (min)	Cluster / laptop
Ingestion and data-quality audit	0.4	2.6	6.7×
Cleaning and pre-processing	0.6	2.5	4.0×
Task 1 analyses	0.5	2.5	4.6×
Stage 1: features, sets, baselines and models	7.9	30.2	3.8×
Stage 2: features and screening	1.2	5.1	4.1×
Stage 2: cross-validation (100 fits)	37.4	161.9	4.3×
Stage 2: ablation, calibration, refit, 2024 test, head-to-head	24.5	84.8	3.5×
Total	**73**	**290**	**4.0×**

Table 18. The same notebook on the laptop and on the Dataproc cluster: minutes per stage (t40_stage_timings.csv of each run).

Result	Laptop	Cluster
Cleaning, Task 1 tables and Stage 1 data sets (9 tables)	identical	identical (largest difference 1e-12)
Stage 1 logistic model: 2024 macro PR-AUC	0.4462	0.4462
Cross-validation: best setting of all four grids	—	same settings
Cross-validation: random-forest log loss	0.0319	0.0319
Final random forest on 2024: PR-AUC at 1 / 5 / 20 mm	0.667 / 0.465 / 0.185	0.666 / 0.465 / 0.189
Head-to-head at 5 mm: final vs Stage 1 PR-AUC	0.459 vs 0.418	0.460 vs 0.418

Table 19. Results of the two runs. Everything deterministic is identical; the tree models differ only in the third decimal because their row subsampling depends on partitioning.

The only Task 1 value that differs is the 99th-percentile event depth (by 0.26 mm), computed with Spark's approximate percentile, whose answer depends on how the data are partitioned. The cluster run therefore confirms that the pipeline is portable: the same code gives the same conclusions on one machine and on a YARN cluster.

8.7 Verification

- **Metrics cross-checked against MLlib:** at 5 mm on 2023 the binned average precision (0.4448) and ROC-AUC (0.9268) agree with BinaryClassificationEvaluator (0.4417, 0.9269). The Stage 2 metrics use the evaluator directly.
- **Leakage checks passed:** targets and features recomputed in pandas from truncated data match Spark for both stages; no anchor with a missing future slot entered the feature table; and both stages purge the train/test boundary (Stage 2 drops the 126 end-of-2023 rows whose target window reaches into 2024).
- **Reproducible:** the pipeline reproduces the group's earlier script runs exactly for Task 1 and the logistic model. Spark's tree models shift in the third decimal between runs because their row subsampling depends on data partitioning.

9. Interpretation & Insights

9.1 What drives the forecast

Feature	Group	Importance	Plain-English explanation
thr	mandatory input	0.193	The threshold itself: the probability falls steeply as the asked-for amount rises.

Feature	Group	Importance	Plain-English explanation
log_thr	mandatory input	0.167	Log of the threshold, the same input on a scale the trees split on more easily.
r5	recent rain	0.140	Rain at the station in the last 5 minutes. Rain now is the best predictor of rain in the next half-hour.
min_since_rain	recent rain	0.074	Minutes since the gauge last saw rain (capped at 24 h). A long dry spell makes rain in the next 30 minutes unlikely.
r15	recent rain	0.068	Rain in the last 15 minutes: is it a passing shower or a storm?
outer_mean	neighbours	0.065	Mean 30-minute rain at gauges within 10 km: is a storm nearby?
r30	recent rain	0.040	Rain in the last 30 minutes.
inner_mean	neighbours	0.037	Mean 30-minute rain at gauges within 5 km.
outer_max	neighbours	0.034	The wettest gauge within 10 km.
inner_max	neighbours	0.032	The wettest gauge within 5 km.

Table 20. Most important features of the final model (mean decrease in impurity) with a plain-English reading.

The feature-group ablation agrees (GBT retrained without each group, validated on 2023): removing the station's **recent-rain** features raises the log loss by +7.8 %, removing the **neighbours** by +2.9 % and the **island-wide** signal by +0.7 %, whereas removing time costs +0.2 % and removing latitude/longitude nothing (the station index already carries location). Model A, with no rain observation at all, is +87.3 % worse.

9.2 Where the model works and where it struggles

- **Best in the Inter-monsoon (Oct-Nov)** (Brier skill 0.33 at 5 mm) and weakest in the NE Monsoon (Dec-Mar) (0.27): large, organised monsoon rain is well signalled by neighbours, scattered convective showers less so.
- **Weakest around 12:00** (Brier skill 0.11) and strongest at 23:00 (0.48): afternoon convective storms often start with little warning in the gauge record, whereas night-time rain is usually part of a system already visible at other gauges.
- **Heavy thresholds are hardest:** at 20 mm (0.1 % of half-hours) PR-AUC is 0.185. ROC-AUC stays high, so the model still ranks the risky half-hours well, but it cannot say *how* heavy a storm will be.
- **A single forecast can miss.** A storm can stop or start without warning. The probabilities are judged over many forecasts, which is what the reliability diagrams do.

9.3 Spark concepts from the course in this pipeline

Narrow and wide transformations. A narrow transformation keeps every partition separate. A wide one must bring rows with the same key together and therefore **shuffles** data between executors. The cost of this pipeline is in its wide steps, so they were designed to shuffle as little as possible:

Step	Spark operation	Type	Data movement
Typing, timestamp snap, range check, reading type	select, withColumn, filter	narrow	none
De-duplicate station-slots	groupBy(station, slot).agg(max(struct(update, value)))	wide	one shuffle; sort-based aggregate within partitions (no global sort)
Station-days, island-days, monthly and annual totals	groupBy aggregations	wide	partial aggregation, then a shuffle of small partial results
Rolling rain windows, time since rain, storm trend	Window.partitionBy(station).orderBy(time).rangeBetween	wide	shuffle by station, sort within partitions
Neighbour rings and gauge-fault checks	join with a broadcast station-pair table	narrow on the large side	the small table is copied; the 48 M rows stay put
Station features × neighbour features	join on (station, slot)	wide	sort-merge join: both sides shuffled
Hash sampling, threshold expansion	filter, explode	narrow	none

Table 21. Narrow and wide transformations in the pipeline.

Join strategies and adaptive query execution. Spark broadcasts the smaller side of a join when it is below the broadcast threshold (10 MB) and otherwise uses a sort-merge join. The physical plans (Section 12.4 of the notebook) confirm the design: the station-pair table is joined with a **BroadcastHashJoin**, and Spark chooses the same **BroadcastHashJoin** even without our hint, while the join of the two large feature tables is a **SortMergeJoin** with

both sides shuffled. **Adaptive query execution** re-plans at run time: after a grouped aggregation ran, its final plan shows the 64 post-shuffle partitions merged by AQEShuffleRead coalesced, because the real data needed far fewer.

```
-- small x large, F.broadcast hint (neighbour rings):  +- BroadcastHashJoin [nb_id], [nb_id],
Inner, BuildRight, false
-- small x large, no hint (Spark decides):  +- BroadcastHashJoin [nb id], [nb id], Inner,
BuildRight, false
-- large x large (station features x neighbour features):  +- SortMergeJoin [station_id,
slot_epoch], [station_id, slot_epoch], LeftOuter
-- grouped aggregation after it ran (AQE final plan):  *(2) HashAggregate(keys=[year],
functions=[sum(rain_mm)], output=[year, mm])
```

How MLlib trains these models across executors. The logistic regression is fitted with L-BFGS: in every iteration each partition computes the loss and gradient of its own rows, and **treeAggregate** adds the partial results up in a tree before the driver takes the step. This is the same pattern as the distributed ridge regression in the lectures, where each partition contributes $X^T_pX_p$ and $X^T_py_p$. The random forest and the GBT bin each feature (maxBins = 128). Each partition builds histograms of the label per bin for every tree node being split, the histograms are aggregated, and the driver chooses the best split. The CrossValidator fits four models in parallel (parallelism = 4) on the year-blocked folds, and the sample weights reach the learners through weightCol.

Cloud optimisation. The lectures recommend Spot VMs, local SSDs and autoscaling. We did not use them, for reasons specific to this project: secondary (Spot) workers would count against the same 12-vCPU quota as the primary workers; one local SSD is 375 GB, more than the project's 250 GB SSD quota; and autoscaling is not intended for a cluster that keeps its data on HDFS, as the complete run did. Parquet's column layout gives the benefit the lectures attribute to column-based storage, and its year partitioning lets Spark skip whole years (partition pruning, used by the weak-scaling runs).

10. Limitations, Risks & Future Work

Limitation	Detail
One test year	2024 is the only untouched year; eight years are too short to separate climate trends from variability.
Rare heavy rain	Only a few hundred training positives above 30 mm, so probabilities for very heavy thresholds are the least certain.
Gauge data only	The model sees only rain already falling on a gauge. Radar, satellite or wind fields would show storms before they reach any gauge.
Grid edges	Some best settings (RF trees and depth, GBT depth and iterations) lie on the edge of their search grid; a wider grid could improve them further.
Cluster size	The 12-vCPU quota limited the cluster to 2–4 small workers, so the cluster ran slower than the laptop; the submitted model is the laptop-trained one (the complete cluster run reproduced its results).
Timestamp meaning	The data documentation does not say whether a stamp marks the start or end of its interval; we chose the reading under which no future information can leak, at worst labelling the prediction time five minutes late.

Table 22. Data and model limitations.

Future work: add radar or satellite inputs; tune beyond the grid edges; run the model training on a larger Dataproc cluster with local SSDs; try the RAPIDS accelerator for Apache Spark on a GPU cluster, which runs filters, projections, sorts and joins on the GPU's columnar memory; and recalibrate the model each season as the gauge network changes.

11. Conclusion

Starting from 7.5 GB of raw five-minute readings, a carefully built Spark pipeline (explicit schema, Parquet layers, neighbour-based gauge-fault detection and completeness rules) produced a clean, trustworthy record of Singapore's rainfall. Task 1 showed two wet seasons, two daily rhythms, a 2× west–east gradient and dry years caused by fewer events. Task 2 turned those findings into features and delivered a single Spark MLlib model for any station, time and threshold: a random forest chosen by year-blocked cross-validation that reaches a PR-AUC of 0.465 at 5 mm on the untouched 2024 year, beats the interpretable logistic baseline at every threshold on identical rows, and gives probabilities that are well calibrated on average.

The main lesson is that **data quality and the information boundary matter more than the model family**: catching faulty gauges changed Task 1's conclusions, and the leakage checks caught a look-ahead that no metric would have flagged. Once the features were right, the choice between logistic regression, random forest and boosting moved the result by far less than the features themselves.

Declaration of AI usage – Generative AI (Claude, Anthropic) was used as a coding assistant for the PySpark pipeline, the Dataproc benchmark, figures and debugging; to discuss design choices (the threshold-as-input formulation, label-independent sampling, leakage checks, evaluation metrics); to review and consolidate the group's two pipelines into one notebook; and to draft the README and this report from the pipeline's outputs. All numbers in this report are produced by the submitted notebook. The authors reviewed the code, the results and the text, and take responsibility for the submission.

References. Data.gov.sg, Historical Rainfall across Singapore (collection 2279). Apache Spark 3.5.5 MLlib Guide. Runfola et al. (2020), geoBoundaries, PLOS ONE 15(4) (Singapore coastline, ODbL). Mann (1945), Kendall (1975), Sen (1968) for the trend test. Brier (1950) for the Brier score.

Submission. Source code: DSA5208_Project2.ipynb (one notebook, run top to bottom, every output kept) with README.md and environment.yml. Trained model: models/task2_final_rf/ (Spark PipelineModel + calibrator + feature_spec.json); Stage 1 baseline in models/task2_baseline_lr/. Cloud benchmark: cloud/ (job script and gcloud commands). Figures and tables: outputs/.